How to Use the Lab Series

Dr. Rochdi Boudjehem

2025-10-24

Purpose

This guide explains how the lab handouts are structured and how you should use them week-to-week. It applies to:

- Prerequisite labs (C pointers, memory model, concurrency basics)
- Core HPC labs (toolchain baseline, OpenMP, MPI, CUDA, etc.)

Use it as a checklist before, during, and after each lab to submit high-quality work.

```
Note:
- The prerequisite labs are facultative and are not mandatory.
- You are free either to complete them for practice or skip them if you feel confident.
- They are designed to make sure you have the foundational C skills needed for HPC program
ming. They aim to help you get comfortable with low-level concepts like pointers, memory m
anagement, and concurrency before diving into parallel programming paradigms.
```

---

1) Anatomy of a Lab Handout

Most lab documents follow the same sections so you can navigate quickly:

- Header (front matter)
  - Title, author, date. Sometimes a short summary.
- Introduction / Objectives
  - What concepts/skills you'll practice and why they matter for HPC.
- Environment & Build
  - OS/toolchains to use and minimal build/run commands (Windows cmd.exe first; WSL/Linux optional).
  - For Windows, we prefer MinGW-w64 GCC; for MPI, MS-MPI (mpiexec) when available.
- Exercises / Tasks
  - Numbered, incremental tasks with small deliverables and observable outcomes.
  - Often include a starter C template to accelerate your work and standardize outputs.
- Starter Templates (code)
  - Drop-in snippets you can copy to a file (e.g., `lab_xxx.c`).
  - Aim to modify minimally: keep I/O format similar to ease grading.
- Build/Run Examples
  - Short commands to compile and execute. Adjust paths if your installation differs.
- What to Submit
  - Required files and naming convention for your submission folder/zip.
  - Usually includes your source(s), build commands, and a README with results/discussion.
- Grading Rubric

  - How points are allocated (correctness, performance analysis, clarity, hygiene).
- Troubleshooting Tips
  - Common pitfalls (linker errors, PATH, OMP/MPI issues), with fixes.
- Optional Extensions
  - Stretch goals for extra practice or deeper exploration.

---

2) Before the Lab (Preparation Checklist)

- Read the Introduction/Objectives to know what to focus on.
- Ensure toolchains are installed and on PATH (Windows):
  - MinGW-w64 GCC (gcc, g++)
  - MS-MPI (mpiexec) for MPI labs; CUDA tools for GPU labs (if applicable)
- Create a working directory per lab (e.g., `lab_prereq01_pointers_<YourName>`).
- Skim the starter templates and note what input/output is expected.

---

3) During the Lab (Working Effectively)

- Start from the given starter file and compile early to catch toolchain issues.
- Complete exercises in order; verify outputs against the task prompts.
- Keep your code clean:
  - Enable warnings (`-Wall -Wextra`) and fix or justify them.
  - Print clear, minimal results as required by the handout.
- Record measurements (time, speedup, GB/s, GFLOP/s) where asked.
- If something fails, check the Troubleshooting section first, then ask.

---

4) After the Lab (Submission Package)

Follow the handout's "What to Submit" section. Typical contents:

- Source files you wrote or modified (e.g., `lab.c`, `mpi_lab.c`).
- Build commands (in a `build.cmd` or in your README).
- README.md with:
  - Platform summary (OS, CPU/GPU model, RAM)
  - Toolchain versions (gcc, mpiexec, nvcc if used)
  - Results: tables/plots requested in the lab
  - Short discussion: what you observed and why

Naming convention (unless specified otherwise): - `lab_<topic>_<YourName>` or as instructed by the lab.

---

5) Running on Windows vs WSL/Linux

- Windows cmd.exe is the primary target for our course.
- WSL/Linux is optional and useful for POSIX-specific content (e.g., pthreads).
- If you switch platforms, note differences in your README (e.g., include flags like `-pthread` on Linux).

Optional commands for reference (do not paste blindly; adapt to your setup):

Windows (cmd.exe):

```
set CC=gcc
%CC% -O2 -Wall -Wextra -pedantic lab.c -o lab.exe
lab.exe
```

OpenMP (cmd.exe):

```
set OMP_NUM_THREADS=8
%CC% -O3 -fopenmp lab_omp.c -o lab_omp.exe
lab_omp.exe
```

MS-MPI (cmd.exe):

```
set MSMPI_INC=C:\Program Files (x86)\Microsoft SDKs\MPI\Include
set MSMPI_LIB64=C:\Program Files\Microsoft MPI\Lib\x64
%CC% -O3 mpi_lab.c -I"%MSMPI_INC%" -L"%MSMPI_LIB64%" -lmsmpi -o mpi_lab.exe
mpiexec -n 4 .\mpi_lab.exe
```

WSL/Linux (optional):

```
gcc -O2 -Wall -Wextra -pedantic lab.c -o lab
./lab
```

---

6) Measuring and Reporting Performance

When a lab asks for performance:

- Run multiple times; report best and average if the handout asks.
- Include exact problem sizes (e.g., N=1e8), timing method, and unit conversions.
- Show how you computed metrics such as:
  - Speedup S(T) = T1 / TT
  - Efficiency E(T) = S(T) / T
  - Throughput GB/s ≈ bytes_moved / time
  - GFLOP/s (or MFLOP/s) as appropriate
- State compiler flags (e.g., `-O0`, `-O2`, `-O3`, `-march=native`).

---

7) Troubleshooting Quick Wins

- gcc not found: ensure MinGW-w64 `...\mingw64\bin` is on PATH; open a fresh cmd.
- MS-MPI link errors: verify SDK installed; correct `MSMPI_LIB64` path (x64 vs x86); quote paths.
- mpiexec runtime errors: try full path to mpiexec; confirm firewall prompts if any.
- OpenMP not working: compile with `-fopenmp`; set `OMP_NUM_THREADS`.
- Large allocations fail: reduce N or increase RAM/swap; check for `NULL`.
- Timing noisy: close background apps; plug in laptop power; run multiple trials.

---

8) Using the Provided Project Files

The repository includes helpful samples and VS Code configs: - `PW/C/threads/` and `PW/C/MPI/` contain example C programs and build scripts. - `.vscode/tasks.json` where present lets you build from VS Code without typing commands. - Batch files like `compile_msmpi.bat` show working MPI link lines on Windows.

You may copy patterns from these to your lab folder, then customize minimally.

Happy hacking — and measure twice before you run fast! 🚀